

JWT Paso a Paso

La Guía Definitiva - Proyecto Cine V2

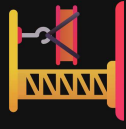
De Inseguro a Fort Knox en 6 Fases

Empezar el viaje →

Mapa de Ruta

Construiremos la seguridad capa por capa, entendiendo el PORQUÉ de cada pieza.

1. Fase 1: Login Básico (Sin seguridad real)
2. Fase 2: BCrypt (Cifrado de contraseñas)
3. Fase 3: JWT (El Token firmado)
4. Fase 4: El Filtro (Interceptando peticiones)
5. Fase 5: UserDetailsService (El Traductor)
6. Fase 6: Protección (Cerrando las puertas)



FASE 1

Login Básico (Inseguro)

1.1 El Objetivo Inicial

Antes de pensar en tokens o cifrados complejos, necesitamos responder a la pregunta más básica:

¿Existe un usuario con este email y esta contraseña en mi base de datos?

Si no somos capaz de validar esto, el resto da igual.

1.2 DTOs: La Regla de Oro

Nunca, JAMÁS, recibas una Entidad JPA en un controlador.

✗ Mal

```
1 @PostMapping("/login")
2 public Usuario login(@RequestBody Usuario u) {
3     // Expone campos internos (id, fechaAlta...)
4     // Obliga a instanciar una Entidad
5 }
```

✓ Bien (DTO)

```
1 // 📁 Instala en 'dtos.login'
2
3 // 📁 ENTRADA
4 public record LoginRequestDTO(
5     String email,
6     String password
7 ) {}
8
9 // 📁 SALIDA
10 public record LoginResponseDTO(
11     String email,
12     String message
13 ) {}
```



1.3 Records

Java 16+

```
public record Auto(String  
marca) {}
```

- ✓ Constructor Canónico
- ✓ Accesor automático (`auto.marca()`)
- ✓ `equals()` / `hashCode()`
- ✓ `toString()`
- 🔒 Inmutables (`final`)

¿Dónde usarlos?

- DTOs (Data Transfer Objects)
- Respuestas JSON de API
- Claves en Mapas (HashMaps)
- Streams (Tuplas temporales)

Documentación Oficial

Oracle Java SE 17: Record Classes

1.4 La Filosofía del Record

¿Por qué Java necesitaba esto?

El Viejo JavaBean (50 líneas)

Getters, Setters, Constructor vacío, Constructor lleno, equals, hashCode, toString...

Solo para guardar 2 Strings.



Java Record (1 línea)

Todo lo anterior, gratis y sin errores humanos.



Tuplas con Nombre

No es una lista anónima `[A, B]`.

Es un `LoginRequest`. Tiene significado semántico.



Inmutabilidad (Safety)

Nadie puede cambiar los datos por error a mitad de camino. Lo que nace, muere igual.



Portador de Datos

Está diseñado para Mover Datos, no para tener lógica de negocio compleja.

1.5 DTOs: Mapa Mental

¿Qué datos viajan en cada momento?

Explicar que hay que crear 3 DTOs en el paquete `dtos`` :

1 LoginRequestDTO

👉 Entrada del Login

```
record LoginRequestDTO(  
    String email,  
    String password  
) {}
```

2 LoginResponseDTO

👉 Salida del Login

```
record LoginResponseDTO(  
    String email,  
    String message,  
    String token  
) {}
```

3 RegisterRequestDTO

👉 Entrada del Registro

```
record RegisterRequestDTO(  
    String email,  
    String password  
) {}
```


1.6 Servicio: Login en Texto Plano

En `UsuarioService.java` :

```
1 public LoginResponseDTO login(LoginRequestDTO request) {
2     // 1. Buscar por email
3     Usuario usuario = repository.findByEmail(request.email())
4         .orElseThrow(() -> new EntityNotFoundException("Usuario no encontrado"));
5
6     // 2. Comparar contraseña (ERROR GRAVE DE SEGURIDAD AQUÍ)
7     if (!usuario.getPassword().equals(request.password())) {
8         // throw new BadCredentialsException("Contraseña incorrecta");
9         throw new RuntimeException("Contraseña incorrecta"); // Cambiaremos a BadCredentialsException con Spring S
10    }
11
12    // 3. Devolver DTO (NO entidad)
13    return new LoginResponseDTO(
14        usuario.getEmail(),
15        "Login exitoso (Inseguro)"
16    );
17 }
```

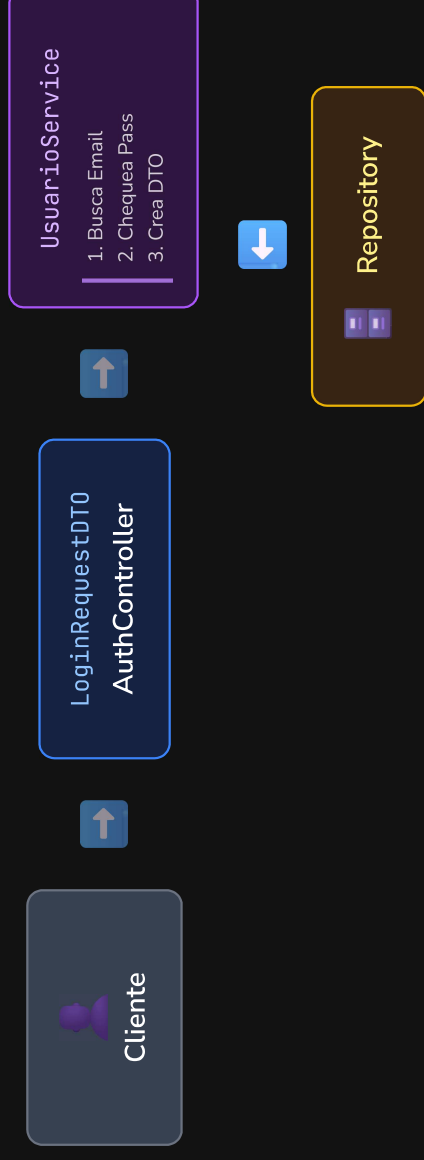
💀 ¡ESTO ES INSEGURO! 💀

1.7 El Controlador: AuthController

Exponemos la lógica al mundo exterior mediante una API REST.

```
1 @RestController
2 @RequestMapping("/api/v1/auth")
3 public class AuthController {
4
5     private final UsuarioService usuarioService;
6
7     @PostMapping("/login")
8     public ResponseEntity<LoginResponseDTO> login(@RequestBody LoginRequestDTO loginRequest) {
9         return ResponseEntity.ok(usuarioService.login(loginRequest));
10    }
11 }
```

1.8 Flujo de una Petición Exitosa ✓



✓ 200 OK

Si el servicio devuelve un usuario, todo ha ido bien.

✗ Errores (401/404)

Si el servicio lanza excepción, nuestro

``GlobalExceptionHandler`` las captura y devuelve JSON de error.

1.9 Estado de la Cuestión

Lo que funciona

- El usuario puede loguearse en el sistema.
- Validamos reglas de negocio (existe/ho existe).
- Devolvemos códigos HTTP correctos (200, 401, 404).

Lo que falta

- Las contraseñas están EN CLARO en la Base de Datos.
- Si roban la BD, tienen acceso a TODO.
- Exposición de seguridad crítica.

FASE 2

BCrypt (Seguridad Real en BD)

2.1 El Problema: Texto Plano y Cifrado Reversible

¿Por qué no basta con ocultarlo?

1. Texto Plano

Guardar `1234` en la BD sin transformar.

- Cualquier admin de BD puede verla.
- Si sufres un ataque (SQLi), roban las credenciales.
- Responsabilidad legal total.

Cifrado (Encryption)

-  Bidireccional: Se puede revertir si tienes la clave (Key).
-  El Riesgo: Si el atacante roba la clave (o un empleado descontento la tiene), descifra TODAS las contraseñas al instante.

"HoLa" +  → "Xy7#b" → "HoLa"

 OWASP Password Storage Cheat Sheet

2.2 La Solución Real: Hashing

Matemáticas sin retorno



Hashing

-  Unidireccional: NO tiene vuelta atrás.
-  No existe "descifrar" un hash.
-  *Ejemplo:* Contraseñas. Solo necesito saber si coinciden.

"Hola" → "5d4140..." → ?



Analogía del Batido

Puedes hacer un batido de una fresa (Hash).

Pero no puedes sacar la fresa del batido.

Si vuelvo a batir la misma fresa, obtengo el mismo batido.
Así verificamos el login.

2.3 Anatomía de BCrypt

¿Qué significan esos caracteres raros?

Cuando ves esto en la base de datos:

```
$2a$10$EixZaYVK1fsbw1ZfbX30XePaWrn96p...
```

No es cadena aleatoria. Tiene una estructura precisa:

 \$2a\$ VERSIÓN	 10 COST FACTOR	EixZa... SALT (22 CHARS)
Identificador del algoritmo (BCrypt).	2^10 iteraciones. Define la lentitud.	Aleatoriedad pura generada por usuario.

Los últimos 31 caracteres son el hash real resultante.

2.4 El Poder del SALT 🧂

¿Por qué dos contraseñas iguales tienen Hash distinto?

Si Pepe y Maria tienen la clave ``123456`` :

✗ SHA-256 (Sin Salt / Salt fijo)

```
Pepe: 8d969e...  
Maria: 8d969e...
```

Peligro: Rainbow Tables buscan "8d969e" y saben que es "123456".

Conclusión: El SALT es un valor aleatorio que se concatena a la contraseña antes de hashear. BCrypt lo genera y lo guarda dentro del propio string final.



✓ BCrypt (Auto-Salt)

```
Pepe: $2a$10$Xy9Z... (Salt A) → a8f9...  
Maria: $2a$10$Bk2L... (Salt B) → c4d2...
```

Misma password, hashes radicalmente distintos.

¡Rompe las Rainbow Tables!

2.5 Cost Factor (Work Factor)

La defensa contra la Fuerza Bruta

La velocidad es enemiga de la seguridad en contraseñas. Si un atacante puede probar 1.000.000 claves/segundo, descubrirá la clave.

El Cost Factor (10) hace que el algoritmo sea intencionalmente LENTO.

- Cost 10: ~100ms por verificación.
- Si el atacante quiere probar todo el diccionario, tardará años en lugar de minutos.

El Equilibrio

- Muy bajo (8): Inseguro (demasiado rápido de romper).
- Muy alto (16): Mal UX (el usuario espera 5 segundos para loguear).
- Estándar (10-12): Perfecto (seguro y usuario no lo nota).

2.6 Flujo: Registro vs Login



1 REGISTRO

Usuario envía "hunter2"



BCrypt:
1. Genera Salt aleatorio
2. Hash("hunter2" + Salt)
3. Resultado: `\$2a\$10\$Salt...Hash`



Guardar en BD

2 LOGIN

Usuario envía "hunter2"



passwordEncoder.matches():
1. Lee el Salt del Hash de la BD
2. Hash("hunter2" + SaltLeído)
3. ¿Coincide con el de la BD?



✓ TRUE / ✗ FALSE



2.7 Paso 1: Dependencia

Añadimos `spring-boot-starter-security`.

¿Qué nos regala esta dependencia?

- Auto-Configuración: Cierra todos los endpoints por defecto.
- Filtros de Seguridad: Crea la cadena de filtros (SecurityFilterChain).
- Cifrado: Nos da acceso a `BCryptPasswordEncoder`.

¡Cuidado! Efecto Inmediato

En cuanto añades esto y reinicias, TODAS tus rutas devuelve 401 Unauthorized.

Spring protege todo por defecto hasta que le digas lo contrario.

2.8 Paso 2: Configuración (Beans)

Es buena práctica crear un paquete `security` para organizar estas clases.

 `src/main/java/.../security/PasswordEncoder.java`

1 Dependencia (pom.xml)

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-security</artifactId>
4 </dependency>
```

2 Configuración (Java)

```
1 @Configuration
2 public class PasswordConfig {
3
4     @Bean
5     public PasswordEncoder passwordEncoder() {
6         return new BCryptPasswordEncoder();
7     }
8 }
```

Al definir este `@Bean`, podremos inyectar `PasswordEncoder` en cualquier Servicio.

2.9 Paso 3: Implementación

Servicio y Controlador (Registro + Login)

UsuarioService.java

```

1  @Service
2  @RequiredArgsConstructor
3  public class UsuarioService {
4
5      private final UsuarioRepository repo;
6      private final PasswordEncoder encoder; // I
7
8      // ♦ REGISTRO
9      public void register(RegisterRequestDTO req
10         Usuario u = new Usuario();
11         u.setEmail(req.email());
12         // 🔒 CIFRAR ANTES DE GUARDAR
13         u.setPassword(encoder.encode(req.password
14         u.setRol("USER");
15         repo.save(u);
16     }
17
18     // ♦ LOGIN

```

AuthController.java

```

1  @RestController
2  @RequestMapping("/auth")
3  @RequiredArgsConstructor
4  public class AuthController {
5
6      private final UsuarioService service;
7
8      @PostMapping("/register")
9      public ResponseEntity<RegisterResponseDTO> r
10         @RequestBody RegisterRequestDTO req
11     ) {
12         service.register(req);
13         return ResponseEntity
14             .status(HttpStatus.CREATED)
15             .body(new RegisterResponseDTO(req.e
16     }
17
18     @PostMapping("/login")

```

2.10 Paso 4: Abrir las Puertas

SecurityConfig

Aunque tengamos todo el código listo, si probamos ahora fallará (401 Unauthorized). Spring Security ha cerrado todo por defecto. Tenemos que decirle qué rutas son públicas.

```
1 @Configuration
2 @EnableWebSecurity
3 public class SecurityConfig {
4
5     @Bean
6     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
7         http
8             .csrf(csrf -> csrf.disable()) // Necesario para APIs REST (Stateless)
9             .authorizeHttpRequests(auth -> auth
10                 // 🔒 PERMITIR ACCESO PÚBLICO A /auth/** (Registro y Login)
11                 .requestMatchers("/auth/**").permitAll()
12                 // 🔒 TODO LO DEMÁS: Requiere autenticación
13                 .anyRequest().authenticated()
14             );
15
16         return http.build();
17     }
18 }
```



FASE 3

JWT (JSON Web Token)

3.1 El Concepto: "Tu Carné de Identidad Digital"

Imagina que JWT es como un DNI o Pasaporte que te expide el servidor.

1. Login Exitoso

El servidor comprueba tu contraseña y user, y te imprime un carné (Token) firmado.



2. Peticiones Futuras

Para entrar al club, ya no dices tu contraseña. Solo enseñas el carné.



eyJhbGciOiJIUzI...

"Este usuario es PEPE

Es ADMIN

Válido hasta las 18:00"

(Firma del Servidor)

3.2 Anatomía de un JWT

Un string largo con 3 partes separadas por puntos

```
`. ` : `aaaaaa.bbbbbbb.cccccc`
```

1. Header: "Soy un JWT y uso el algoritmo HS256".
2. Payload (Cuerpo): "Me llamo pepe@gmail.com, soy ADMIN y caduco en 1 hora".
3. Signature (Firma): Sello de lacre criptográfico.

3.3 La Firma (Signature)

Esto es lo vital. La firma se calcula así:

```
`HMACSHA256(Header + Payload, SECRETO_DEL_SERVIDOR)`
```

Si el usuario (hacker) intenta cambiar "soy ADMIN" en el Payload:

1. La firma matemática ya no coincidirá.
2. El servidor rechazará el token.

Solo el servidor (que tiene el SECRETO) puede crear tokens válidos.

3.4 La Herramienta: JWT

Para trabajar con tokens, usamos la librería JWT (Java JWT).

```
1 <dependency>
2   <groupId>io.jsonwebtoken</groupId>
3   <artifactId>jjwt-api</artifactId>
4   <version>0.11.5</version>
5 </dependency>
6 <dependency>
7   <groupId>io.jsonwebtoken</groupId>
8   <artifactId>jjwt-impl</artifactId>
9   <version>0.11.5</version>
10  <scope>runtime</scope>
11 </dependency>
12 <dependency>
13   <groupId>io.jsonwebtoken</groupId>
14   <artifactId>jjwt-jackson</artifactId>
15   <version>0.11.5</version>
16   <scope>runtime</scope>
17 </dependency>
```

3.5 Arquitectura de Seguridad 🏛️

El Trío Dinámico

Todo lo relacionado con seguridad debe ir en su propio paquete:

📁 `src/main/java/.../security/``



SecurityConfig

El Portero.

Decide quién entra y quién no (Rutas públicas vs privadas).



PasswordConfig

El Candidato.

Configura BCrypt para cifrar las contraseñas.



JwtUtil

La Fábrica.

Genera, firma y valida los Tokens (DNI).

NEW

3.6 Preparando el Terreno: Properties

En `application.properties``, definimos la estructura, pero no el secreto.

```
1 # 1. El secreto se inyecta desde una Variable de Entorno
2 # Nunca escribas la clave real aquí
3 jwt.secret=${JWT_SECRET}
4
5 # 2. Tiempo de expiración (1 hora)
6 jwt.expiration=3600000
```

En tu IDE (IntelliJ / VSCode)

Tienes que ir a Edit Configurations -> Environment Variables
y añadir:

```
`JWT_SECRET=MiClaveSuperSecreta12345 ...`
```

Por qué hacemos esto?

Para no subir accidentalmente nuestras contraseñas a GitHub.

3.7 JwtUtil: La Implementación

```
1 @Component
2 public class JwtUtil {
3
4     @Value("${jwt.secret}")
5     private String secret; // "MiClave..." (String)
6
7     @Value("${jwt.expiration}")
8     private long expirationTime;
9
10    private SecretKey key; // Objeto Criptográfico Real
11
12    // ⚡ SE EJECUTA DESPUÉS DEL CONSTRUCTOR
13    @PostConstruct
14    public void init() {
15        // Transformamos la password de texto a una llave HMAC-SHA compatible
16        // ¿Por qué aquí? Porque en el constructor 'secret' aún es null.
17        this.key = Keys.hmacShaKeyFor(secret.getBytes());
18    }
19
20    public String generateToken(Usuario usuario) {
21        return Jwts.builder()
22            .subject(usuario.getEmail())
23            .claim("role", usuario.getRol())
24            .build();
25    }
26 }
```

3.8 El Builder Pattern (generateToken)

¿Cómo se fabrica un Token paso a paso?

`.builder()`



Abre la caja vacía del Token.

`.subject()`



"Este DNI pertenece a pepe@gmail.com"

`.claim()`



Añade datos extra:

`"role": "USER"`

`.expiration()`



"Caduca en 1 hora". (Si vienes mañana, no entras).

`.signWith(key)`



 **EL PASO CRÍTICO.**

Pone el sello de lacre con tu SECRETO. Sin esto, el token es papel mojado.

`.compact()`



Cierra la caja y te la da comprimida (String ``ey ...``).

3.9 Modificando el Login (UsuarioService)

Ahora el login no devuelve un `Usuario``, devuelve un `LoginResponseDT0`` con el token.

```
1 public LoginResponseDT0 login(LoginRequest req) {
2     Usuario u = repo.findByEmail(req.email())
3     .orElseThrow(() -> new BadCredentialsException("Usuario no encontrado"));
4
5     if (!passwordEncoder.matches(req.password(), u.getPassword()))
6         throw new BadCredentialsException("Contraseña incorrecta");
7
8     // Generamos el pase VIP (Token)
9     String token = jwtUtil.generateToken(u);
10
11     // Devolvemos DT0 con todo
12     return new LoginResponseDT0(
13         u.getEmail(),
14         "Login exitoso",
15         token
16     );
17 }
```

3.10 Checklist de Seguridad

¿Qué hemos conseguido hasta ahora?

1 Usuarios en Base de Datos

2 Contraseñas Cifradas (BCrypt)

3 Generación de Tokens (JWT)

4 Validación Automática (Filtro)

5 Protección de Rutas

Misión de 5 Minutos: Decodificar

¿Qué esconde realmente el token?

1

Haz Login en Postman
``POST /auth/login``

2

Copia el Token generado
(Control+C / Command+C)

3

Ir a [JWT.IO](https://jwt.io) y PEGAR 

TARGET DATA

"sub"	"tu@email.com"
"role"	"USER"
"exp"	1740 ... (Futuro)

 RECUERDA: ¡El token NO oculta los datos!
Cualquiera puede leer esto.